

Can Graphs Help Vision SSMs See Better?

Dhruv Parikh¹, Anvitha Ramachandran¹, Haoyang Fan¹
Mustafa Munir², Rajgopal Kannan³, Viktor Prasanna¹

¹USC ²UT Austin ³DEVCOM ARL Army Research Office, USA
{dhruvash, alramach, haoyangf, prasanna}@usc.edu,
mmunir@utexas.edu, rajgopal.kannan.civ@army.mil

Abstract

Vision state space models inherit the efficiency and long-range modeling ability of Mamba-style selective scans. However, their performance depends critically on the representation of two-dimensional visual features as one-dimensional token sequences. Existing scan operators range from predefined geometric traversals to dynamic coordinate-based samplers that reroute tokens through predicted offsets and interpolation. While effective, these mechanisms primarily adapt paths or sampling locations, rather than explicitly modeling which local patches should exchange information before global state-space mixing. This motivates a simple question: *can graphs help vision state space models see better?* We introduce **GraphScan**, a graph-induced dynamic scanning operator for Vision SSMs. For each token, GraphScan constructs a spatially bounded local graph, learns feature-conditioned affinities with relative positional bias, and produces the output token by one-step message passing over its semantic neighborhood. The resulting tokens are locally grounded before being processed by the selective SSM for global aggregation. GraphScan preserves token count and linear scaling in image size, while replacing coordinate-conditioned interpolation with feature-conditioned semantic routing. Integrated into a hierarchical backbone, **GraphScan-Mamba** achieves state-of-the-art performance among Vision SSMs across image classification, object detection, instance segmentation, and semantic segmentation, with modest computational overhead. Our analysis further shows that GraphScan induces interpretable displacement fields over the token lattice, providing a semantic and spatially grounded view of dynamic scanning. These results suggest that future Vision SSMs should treat scanning not merely as geometric serialization, but as learned local semantic routing before global state-space modeling.

1 Introduction

The design of visual backbones has repeatedly been shaped by how an image is represented before it is processed. Convolutional networks preserve the image as a regular grid [28, 26, 47, 50, 19, 21, 51, 43, 37, 65, 59, 8, 34, 74] while Vision Transformers recast it as a sequence of patches with self-attention for long-range interaction [57, 10, 54, 36, 61, 62, 3, 9, 73, 17, 67, 77, 56, 5, 2, 63]; all-MLP and graph-based variants further show this representation choice is broader than convolution or attention alone [53, 55, 32, 16]. More recently, structured state space models (SSMs) have emerged as efficient sequence mixers with long-range modeling and linear scaling [15, 14, 48, 11, 42, 13, 6, 27], raising a basic question for vision: if the model is fundamentally a sequence processor, what is the right sequence representation for an image?

Vision Mamba backbones expose this question sharply: Mamba-style selective SSMs offer linear-complexity content-dependent sequence modeling [13], but applying them to two-dimensional images requires serializing a feature map into a token sequence—making the scan operator the architectural interface between image geometry and state-space computation. Early Vision Mamba models use bidirectional or two-dimensional selective scans [78, 35], and subsequent work has explored continuous, local-window, atrous, multi-scale, grouped, multi-dimensional, and fractal traversal patterns [72, 22, 41, 45, 44, 29, 52], establishing scanning as a central design lever in Vision SSMs.

Preprint.

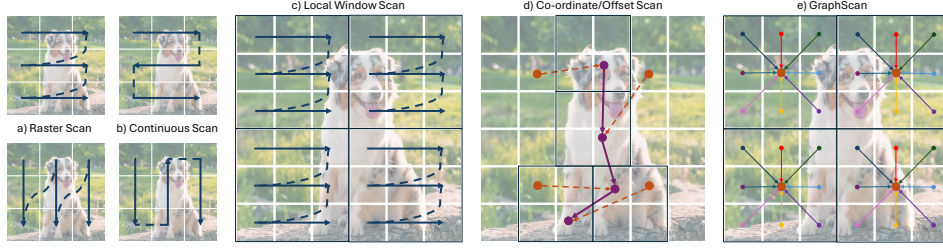


Figure 1: **Comparison of scanning paradigms for Vision SSMs.** Raster and continuous scans expose the SSM to a fixed 1D ordering of the 2D lattice; local-window scans restrict it to bounded predefined regions; coordinate/offset scans introduce input dependence by predicting sampling locations and rerouting through geometric displacement. **GraphScan** instead keeps token slots fixed and constructs a local semantic graph around each token, routing nearby patches through learned inter-patch affinities before global selective scan—scanning as local semantic routing rather than path or coordinate selection.

Table 1: Families of scanning mechanisms in Vision SSMs. *Path*: how the 1D order through the lattice is determined. *Adaptive*: whether the operator is input-dependent. *Local mixing*: whether the operator explicitly aggregates a bounded spatial neighborhood before the SSM. GraphScan occupies the only cell that is simultaneously local, input-adaptive, and semantic.

Family	Path	Adaptive	Local mixing	Representative works
Fixed multi-direction	predefined	–	implicit	[78, 35, 72, 41, 29, 52]
Local windowed	predefined	–	fixed window	[22, 45, 44]
Coordinate-offset	raster	learned offsets	4-corner bilinear	[30, 33, 70]
Score / sort	permuted	learned scalar	–	[75, 60, 24, 71]
Recurrence-side	–	varies	varies	[68, 46, 18]
Graph routing (ours)	raster	learned affinities	semantic window	GraphScan

A second wave of methods asks whether scanning can be made adaptive. Some approaches learn spatial partitions or deformable scan paths, while others predict offsets, objectness scores, dynamic state aggregation patterns, or task-specific scan orders [70, 30, 33, 75, 71, 60, 24]. Other works attack the problem from the recurrence side, for example by introducing structure-aware state fusion, non-causal SSD formulations, or hybrid Mamba-Transformer designs [68, 46, 18]. Together, these works reveal two complementary facts: visual SSMs need strong global sequence modeling, but they also need a better way to preserve the local and semantic structure of images before global propagation. Fig. 1 summarizes these scan paradigms and highlights the semantic-routing axis introduced by GraphScan.

Despite this progress, most adaptive scans remain primarily geometric: coordinate-offset methods sample at predicted positions through bilinear interpolation [30, 33], score- and sort-based methods reorder tokens without aggregating local semantic neighborhoods [75, 24], and recurrence-side methods modify the SSM mixer itself rather than the visual tokens entering it [68, 46]. This leaves open a simple possibility: before asking an SSM to propagate information globally, can we first make each token locally and semantically aware?

We answer this question with **GraphScan**, a graph-induced dynamic scanning operator for Vision Mamba. Instead of predicting a continuous sampling coordinate, GraphScan constructs a spatially bounded local graph around each token and learns feature-conditioned affinities with relative positional bias. Each output token is produced by one-step message passing over this semantic neighborhood and then passed to the selective SSM for global aggregation. GraphScan therefore does not replace the SSM with a graph neural network [25, 58, 16]; rather, it uses graph-based routing as a lightweight scan-time operator that prepares locally grounded tokens for the global sequence mixer.

This gives GraphScan a distinct role among existing scan mechanisms: where fixed scans choose a path, deformable scans choose coordinates, and score-based scans choose an ordering, GraphScan chooses a local semantic neighborhood. The distinction matters because the selective SSM compresses visual context along a one-dimensional processing path; if the tokens entering it already encode spatially nearby and semantically relevant neighbors, the global mixer receives a better-conditioned sequence without changing its recurrence. GraphScan is therefore an input-side complement to recurrence-side advances such as SSD, non-causal SSD, and newer Mamba variants [6, 46, 27].

We instantiate GraphScan in a hierarchical Vision Mamba backbone, **GraphScan-Mamba**, on the Mamba-1/S6 selective-scan interface used by the dominant scan-mechanism literature [78, 35, 72, 22, 41, 45, 68, 30, 33], enabling direct comparison with fixed, local, multi-scale, deformable, and dynamic scan baselines. GraphScan is orthogonal to the choice of SSM core: it changes the tokens presented to the mixer, not the mixer dynamics.

We evaluate GraphScan-Mamba under standard vision-backbone protocols (ImageNet-1K classification, COCO detection/instance segmentation with Mask R-CNN, ADE20K segmentation with UperNet [7, 31, 20, 76, 69]), where it achieves state-of-the-art performance among Vision SSM backbones at comparable parameter and FLOP budgets. Ablations isolate the role of semantic routing, neighborhood size, relative positional bias, and pre-SSM insertion, and visualizations confirm that GraphScan induces interpretable displacement fields over the token lattice. Together, these results suggest a new design principle: scanning should not be treated as geometric serialization but as learned semantic routing before global state-space modeling.

2 Related Work

2.1 Vision Backbones and SSMs

Vision backbones span convolutional [37, 65, 74], Transformer [10, 54, 36, 17, 77], MLP-mixing [53], and graph-based [16, 66] families; we refer to App. B for a fuller treatment. State-space sequence models provide efficient long-range mixing with near-linear scaling [15, 14, 48, 11, 42], and Mamba introduced selective state spaces with input-dependent recurrence parameters [13], later extended through structured state space duality and recurrence-level improvements [6, 27]. Our work addresses a complementary question: how should a two-dimensional visual feature map be routed before it is processed by a one-dimensional selective scan?

2.2 Vision Mamba Scans and Dynamic Routing

Vision Mamba models convert feature maps to token sequences using fixed scan patterns—bidirectional, two-dimensional, continuous, local-window, atrous, multi-scale, grouped, or space-filling [78, 35, 72, 22, 41, 45, 44, 29, 52]. Adaptive variants reroute tokens before the SSM through learned partitions [70], predicted continuous offsets with bilinear interpolation [30, 33], content-dependent reordering [75, 60, 24], dynamic state aggregation [71], structure-aware state fusion [68], non-causal SSD [46], or Mamba–Transformer hybridization [18]. GraphScan instead performs one-step semantic message passing on a local graph over visual tokens before the SSM: input-adaptive like dynamic scans, spatially bounded like local scans, and semantic like graph aggregation, while preserving token count and linear scaling.

3 Method

GraphScan is designed around a simple principle: before a one-dimensional selective SSM performs global sequence mixing, each visual token should first be enriched by the local semantic neighborhood from which it arises. In this section, we first review the Vision Mamba selective-scan interface, then reinterpret adaptive scanning as pre-SSM token routing, introduce the GraphScan operator, and finally describe its integration into the GraphScan-Mamba backbone.

3.1 Vision Selective Scan Preliminaries

Let $X \in \mathbb{R}^{H \times W \times D}$ denote an intermediate visual feature map with D channels. We flatten it into a token sequence $\mathbf{X} = [\mathbf{x}_1, \dots, \mathbf{x}_L] \in \mathbb{R}^{L \times D}$ with $L = HW$. Each token i has a discrete lattice coordinate $\mathbf{g}_i \in \{1, \dots, H\} \times \{1, \dots, W\}$ and a normalized coordinate $\mathbf{p}_i \in [-1, 1]^2$. The downstream SSM consumes the flattened sequence in raster order unless otherwise specified. Batch dimensions are omitted for clarity.

Selective SSM interface. A Mamba-style selective SSM maps an input sequence $\mathbf{U} = [\mathbf{u}_1, \dots, \mathbf{u}_L]$ to an output sequence through an input-dependent recurrence [13]. Let N_s denote the SSM state dimension. For the Mamba-1/S6 interface used by most Vision Mamba backbones, each channel maintains an N_s -dimensional recurrent state. At position t , the static transition parameter $\mathbf{A} \in \mathbb{R}^{D \times N_s}$ is combined with input-dependent quantities generated from $\mathbf{u}_t$: a final positive step size $\Delta_t \in \mathbb{R}^D$ and selective vectors $\mathbf{b}_t, \mathbf{c}_t \in \mathbb{R}^{N_s}$. After discretization, this yields effective tensors $\bar{\mathbf{A}}_t, \bar{\mathbf{B}}_t \in \mathbb{R}^{D \times N_s}$, where $\bar{\mathbf{A}}_t$ controls state retention and $\bar{\mathbf{B}}_t$ controls the input write into the state. The recurrent state is $\mathbf{H}_t \in \mathbb{R}^{D \times N_s}$. We use two broadcast operators. For $\mathbf{u}_t \in \mathbb{R}^D$, let $\mathbf{u}_t^{\uparrow s} \in \mathbb{R}^{D \times N_s}$ denote broadcasting over the state dimension, i.e., $[\mathbf{u}_t^{\uparrow s}]_{d,n} = \mathbf{u}_{t,d}$. For $\mathbf{c}_t \in \mathbb{R}^{N_s}$, let $\mathbf{c}_t^{\uparrow D} \in \mathbb{R}^{D \times N_s}$ denote broadcasting over the channel dimension, i.e., $[\mathbf{c}_t^{\uparrow D}]_{d,n} = \mathbf{c}_{t,n}$. The core update is

$$\mathbf{H}_t = \bar{\mathbf{A}}_t \odot \mathbf{H}_{t-1} + \bar{\mathbf{B}}_t \odot \mathbf{u}_t^{\uparrow s}, \quad \mathbf{y}_t = \left\langle \mathbf{H}_t, \mathbf{c}_t^{\uparrow D} \right\rangle_{N_s}, \quad (1)$$

where $\langle \cdot, \cdot \rangle_{N_s}$ denotes contraction over the SSM state dimension, producing $\mathbf{y}_t \in \mathbb{R}^D$. Across the full sequence, the post-discretization tensors have shape $\bar{\mathbf{A}}, \bar{\mathbf{B}} \in \mathbb{R}^{L \times D \times N_s}$, and the output sequence satisfies $\mathbf{Y} \in \mathbb{R}^{L \times D}$.

The key property for our work is selectivity: Δ_t , $\mathbf{b}_t$, and $\mathbf{c}_t$ are generated from the current token $\mathbf{u}_t$ [13]. Therefore, modifying the token sequence before the SSM changes both the values written into the recurrent state and the selective parameters that govern propagation and readout. Mamba-2 and later SSD-style models provide alternative formulations and more efficient algorithms for related state-space sequence transformations [6]; our method targets the input sequence presented to the SSM and is therefore orthogonal to these recurrence-level developments.

Parallel scan computation. Although Eq. (1) is written recurrently, selective scan is computed efficiently using an associative prefix-scan operator [13]. Define the write term $\mathbf{U}_t = \bar{\mathbf{B}}_t \odot \mathbf{u}_t^{\uparrow s}$ and the affine update pair $\mathbf{P}_t = (\bar{\mathbf{A}}_t, \mathbf{U}_t)$. Composing two updates gives $(\mathbf{A}_2, \mathbf{U}_2) \circ (\mathbf{A}_1, \mathbf{U}_1) = (\mathbf{A}_2 \odot \mathbf{A}_1, \mathbf{A}_2 \odot \mathbf{U}_1 + \mathbf{U}_2)$, which is associative. Thus all hidden states can be obtained by a parallel prefix scan over $\{\mathbf{P}_t\}_{t=1}^L$, preserving linear scaling in sequence length while avoiding materializing a dense $L \times L$ mixing matrix.

Unrolled view. Unrolling Eq. (1) from a zero initial state yields $\mathbf{H}_t = \sum_{i=1}^t \mathbf{G}_{t,i} \odot \bar{\mathbf{B}}_i \odot \mathbf{u}_i^{\uparrow s}$ with $\mathbf{G}_{t,i} = \odot_{j=i+1}^t \bar{\mathbf{A}}_j$ the selective survival factor from sequence position i to t . With a nonzero initial state, an additional term $\mathbf{G}_{t,0} \odot \mathbf{H}_0$ appears, where $\mathbf{G}_{t,0} = \odot_{j=1}^t \bar{\mathbf{A}}_j$; for standard visual prefill over a feature map, we use the zero-state form for clarity. For visual features, this one-dimensional path is only an indirect representation of the original two-dimensional structure. A pair of patches can be adjacent in the image but distant in raster order, making the scan/routing operator a critical interface between visual geometry and SSM computation.

3.2 From Geometric Scanning to Semantic Routing

Existing Vision Mamba backbones improve this interface by changing how the image is serialized or resampled before selective scan. Fixed scans choose a deterministic path through the lattice; local scans restrict processing to windows; adaptive coordinate-based scans predict new sampling locations before feeding features into the SSM [78, 35, 22, 30, 33]. Our goal is not to modify the SSM recurrence itself, but to improve the visual token sequence that enters it.

Geometric adaptive scan as local interpolation. Coordinate-based adaptive scans predict an offset $\Delta \mathbf{p}_i$ for each reference position and sample from $\mathbf{p}'_i = \mathbf{p}_i + \Delta \mathbf{p}_i$. The output token is obtained through bilinear interpolation, $\mathbf{x}'_i = \sum_{j \in \mathcal{N}_4(\mathbf{p}'_i)} w_{ij}^{\text{bilin}} \mathbf{x}_j$ with $w_{ij}^{\text{bilin}} \geq 0$ and $\sum_j w_{ij}^{\text{bilin}} = 1$, where $\mathcal{N}_4(\mathbf{p}'_i)$ denotes the four lattice points surrounding the sampled coordinate. This is an effective form of input-dependent local rerouting, but the neighborhood and weights are determined by geometry: the predicted coordinate chooses the four corners and the interpolation rule fixes their weights.

Our view. We instead ask whether adaptive scanning can be made semantic. Rather than predicting where to sample in continuous coordinate space, GraphScan asks which nearby patches should communicate with the current token. It preserves the same interface as prior pre-SSM rerouting operators: the output remains a length- L sequence in the original raster slots. However, the content assigned to each slot is produced by learned message passing over a local graph, with weights determined by inter-patch affinities.

3.3 GraphScan: Local Semantic Routing

GraphScan constructs, for each token, a bounded local graph on the image lattice and performs one step of feature-conditioned message passing [12]. It is deliberately lightweight: the neighborhood size is fixed by a small radius, the attention is local rather than global, and the output token count is unchanged. Fig. 2 illustrates the local affinity construction and message-passing update used to produce the routed token sequence.

Local candidate set. For token i , we define a spatial candidate set $\mathcal{S}_r(i) = \{j : \|\mathbf{g}_j - \mathbf{g}_i\|_\infty \leq r\}$ of size $S = |\mathcal{S}_r(i)| = (2r + 1)^2$, where r is the graph radius. In implementation, boundary tokens use replicate padding through coordinate clamping, so every token has the same S window slots. This avoids introducing artificial zero tokens into the softmax and keeps the operator shape-static.

Feature-conditioned affinities. For each token, GraphScan computes query, key, and value projections $\mathbf{q}_i = \mathbf{x}_i W_q$, $\mathbf{k}_j = \mathbf{x}_j W_k$, and $\mathbf{v}_j = \mathbf{x}_j W_v$, where $\mathbf{q}_i, \mathbf{k}_j \in \mathbb{R}^d$ and $\mathbf{v}_j \in \mathbb{R}^{d_v}$. For $j \in \mathcal{S}_r(i)$,

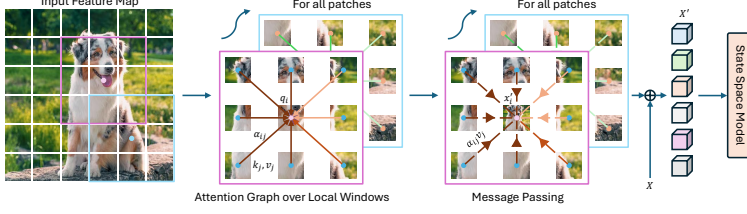


Figure 2: **GraphScan**. Each token attends to a bounded local neighborhood, aggregates semantically weighted neighboring features, and produces a routed token x'_i . The routed feature map X' is then passed to S6.

the affinity score is

$$s_{ij} = \frac{\mathbf{q}_i \mathbf{k}_j^\top}{\sqrt{d}} + b_{\text{rel}}(\mathbf{g}_j - \mathbf{g}_i), \quad \alpha_{ij} = \frac{\exp(s_{ij})}{\sum_{\ell \in \mathcal{S}_r(i)} \exp(s_{i\ell})}. \quad (2)$$

The learnable relative bias b_{rel} is defined over discrete offsets inside the local window. Unless otherwise stated, we use a single affinity head; a multi-head variant follows the standard head-splitting formulation and applies Eq. (2) independently per head, with a shared relative-position bias b_{rel} across heads to keep parameter shapes invariant under head count.

Message-passing update. The routed token is produced by aggregating value features from the learned semantic neighborhood:

$$\mathbf{x}'_i = \mathbf{x}_i + \left(\sum_{j \in \mathcal{S}_r(i)} \alpha_{ij} \mathbf{v}_j \right) W_o, \quad (3)$$

where $W_o \in \mathbb{R}^{d_v \times D}$ projects the aggregated message back to the model dimension. Eq. (3) preserves the original token slot i , but replaces an isolated patch representation with a semantic mixture of nearby visual evidence. The local window specifies only where this evidence may come from; it is not a convolutional kernel. While convolution uses content-independent weights tied to relative offsets, GraphScan recomputes token-specific weights from inter-patch affinities. The operation is therefore related at the primitive level to local neighborhood attention and graph message passing over visual tokens [17, 64, 58, 12, 16, 66], but its architectural role is distinct. GraphScan is not a standalone attention block or Vision-GNN backbone: it is a scan-time routing layer placed immediately before the selective SSM. The routed tokens consequently affect both the values written into the recurrent state and the input-dependent parameters that govern state propagation and readout. GraphScan therefore turns local semantic affinity into a preconditioning step for global state-space mixing (Sec. 3.4); a more detailed comparison to neighborhood attention and Vision-GNN primitives is given in Appendix A.7.

Graph-induced scan field. For analysis and visualization, GraphScan induces an effective routed coordinate by taking the expected source position under the learned affinities, $\hat{\mathbf{p}}_i = \sum_{j \in \mathcal{S}_r(i)} \alpha_{ij} \mathbf{p}_j$, with displacement $\Delta \hat{\mathbf{p}}_i = \hat{\mathbf{p}}_i - \mathbf{p}_i$. Since the weights α_{ij} are nonnegative and sum to one, $\hat{\mathbf{p}}_i$ remains inside the image coordinate domain whenever all $\mathbf{p}_j \in [-1, 1]^2$. This yields a displacement-field visualization analogous to offset-based scans (Fig. 5 in App. D), but the displacement is induced by semantic routing rather than by direct coordinate regression.

Complexity. With fixed radius r , GraphScan has constant graph degree $S = (2r + 1)^2$. The local affinity and aggregation cost scales as $\mathcal{O}(LS(d + d_v))$ up to projection costs, and is therefore linear in the number of image tokens. The routed sequence $\mathbf{X}' = [\mathbf{x}'_1, \dots, \mathbf{x}'_L]$ has the same length as $\mathbf{X}$ and can be passed directly to the selective SSM without changing the scan kernel.

3.4 GraphScan as Selective-Scan Preconditioning

GraphScan can be viewed as a sparse, input-adaptive preconditioner for selective scan. Let $P(\mathbf{X}) \in \mathbb{R}^{L \times L}$ denote the sparse row-stochastic routing matrix induced by Eq. (2), with $P_{ij}(\mathbf{X}) = \alpha_{ij}$ for $j \in \mathcal{S}_r(i)$ and zero otherwise. Writing $M = W_v W_o$, Eq. (3) becomes

$$\mathbf{X}' = \mathbf{X} + P(\mathbf{X}) \mathbf{X} M, \quad \mathbf{Y} = \text{SSM}(\mathbf{X}'). \quad (4)$$

Thus, GraphScan changes the sequence presented to the SSM while leaving the selective-scan recurrence and implementation unchanged. The dependence of P on $\mathbf{X}$ flows through the W_q and W_k projections inside the affinity computation in Eq. (2).

Parameter-modulated selective scan. The effect of Eq. (4) is not limited to changing token values. Since selective SSMs generate Δ_t , $\mathbf{B}_t$, and $\mathbf{c}_t$ from the current token, replacing $\mathbf{x}_t$ with $\mathbf{x}'_t$ also

changes the input-dependent parameters of the recurrence. The routed recurrence is therefore

$$\begin{aligned}\mathbf{H}'_t &= \bar{\mathbf{A}}'_t \odot \mathbf{H}'_{t-1} + \bar{\mathbf{B}}'_t \odot (\mathbf{x}'_t)^{\uparrow s}, \\ \mathbf{y}'_t &= \langle \mathbf{H}'_t, (\mathbf{c}'_t)^{\uparrow D} \rangle_{N_s},\end{aligned}\tag{5}$$

where $\bar{\mathbf{A}}'_t = \bar{\mathbf{A}}_t(\mathbf{x}'_t)$, $\bar{\mathbf{B}}'_t = \bar{\mathbf{B}}_t(\mathbf{x}'_t)$, and $\mathbf{c}'_t = \mathbf{c}_t(\mathbf{x}'_t)$. This is the central reason for placing GraphScan immediately before the SSM: local semantic routing affects both the values written into the recurrent state and the token-dependent parameters that govern propagation and readout.

Exact routed-vs-base decomposition. Let

$$\mathbf{m}_t = \mathbf{x}'_t - \mathbf{x}_t = \sum_{j \in \mathcal{S}_r(t)} \alpha_{tj} \mathbf{x}_j M$$

denote the GraphScan message added to token t . Define the exact parameter differences $\delta \bar{\mathbf{A}}_t = \bar{\mathbf{A}}'_t - \bar{\mathbf{A}}_t$, $\delta \bar{\mathbf{B}}_t = \bar{\mathbf{B}}'_t - \bar{\mathbf{B}}_t$, and $\delta \mathbf{c}_t = \mathbf{c}'_t - \mathbf{c}_t$, where unprimed quantities are generated from the base token $\mathbf{x}_t$. Comparing the routed recurrence in Eq. (5) with the base recurrence in Eq. (1) yields the exact decomposition

$$\delta \mathbf{H}_t = \sum_{i=1}^t \mathbf{G}'_{t,i} \odot \left[\bar{\mathbf{B}}'_i \odot \mathbf{m}_i^{\uparrow s} + \delta \bar{\mathbf{B}}_i \odot \mathbf{x}_i^{\uparrow s} + \delta \bar{\mathbf{A}}_i \odot \mathbf{H}_{i-1} \right],\tag{6}$$

where $\delta \mathbf{H}_t = \mathbf{H}'_t - \mathbf{H}_t$ and $\mathbf{G}'_{t,i} = \odot_{j=i+1}^t \bar{\mathbf{A}}'_j$ is the routed survival factor. The corresponding output difference is

$$\delta \mathbf{y}_t = \langle \delta \mathbf{H}_t, (\mathbf{c}'_t)^{\uparrow D} \rangle_{N_s} + \langle \mathbf{H}_t, \delta \mathbf{c}_t^{\uparrow D} \rangle_{N_s}.\tag{7}$$

Eq. (6) separates three hidden-state effects of GraphScan: direct routed value injection through $\bar{\mathbf{B}}'_i \odot \mathbf{m}_i^{\uparrow s}$, write modulation through $\delta \bar{\mathbf{B}}_i$, and transition modulation through $\delta \bar{\mathbf{A}}_i$. Eq. (7) further adds readout modulation through $\delta \mathbf{c}_t$. Thus, GraphScan does not merely add a local message before the SSM; it changes how the selective scan writes, retains, and reads visual information. The full derivation is given in Appendix A.2.

Local-global kernel view. The direct routed value path in Eq. (6) exposes the local-global structure induced by GraphScan. Define the routed SSM input-to-output operator

$$\mathcal{K}'_{t,i}(\mathbf{z}) = \langle \mathbf{G}'_{t,i} \odot \bar{\mathbf{B}}'_i \odot \mathbf{z}^{\uparrow s}, (\mathbf{c}'_t)^{\uparrow D} \rangle_{N_s}.\tag{8}$$

Using $\mathbf{m}_i = \sum_{j \in \mathcal{S}_r(i)} \alpha_{ij} \mathbf{x}_j M$, the direct routed value contribution becomes

$$\delta \mathbf{y}_t^{\text{route}} = \sum_{j=1}^L \sum_{\substack{i \leq t: \\ j \in \mathcal{S}_r(i)}} \alpha_{ij} \mathcal{K}'_{t,i}(\mathbf{x}_j M).\tag{9}$$

A source patch $\mathbf{x}_j$ is therefore weighted twice: first by a local semantic affinity α_{ij} on the image lattice, and then by the global selective propagation encoded in $\mathcal{K}'_{t,i}$. GraphScan consequently creates a local semantic shortcut before the one-dimensional recurrence, allowing spatially nearby or semantically related patches to influence the selective scan without relying solely on their raster-order distance. The corresponding derivation is given in Appendix A.3.

Further structural consequences. The preconditioner view yields three additional consequences detailed in App. A: a scan-distance attenuation bound that governs how a source patch reaches output t through a routed slot i in terms of $t-i$ rather than raster distance (App. A.4); a modified finite-horizon input-to-state map with cross-token directions induced by the sparse routing matrix $P(\mathbf{X})$ (App. A.5); and baseline containment—setting $W_o = 0$ recovers $\mathbf{X}' = \mathbf{X}$ (App. A.6). GraphScan therefore augments the backbone’s representational family while preserving the original Vision Mamba model as a special case.

Orthogonality to SSM cores. GraphScan is an input-side operator that leaves the discretization rule, selective scan recurrence, SSD formulation, and hidden-state update untouched, so it is complementary to recurrence-side improvements such as SSD, non-causal SSD, and newer Mamba variants [6, 46, 27]. We instantiate it on the Mamba-1/S6 Vision Mamba interface to isolate the effect of scanning and enable direct comparison with existing scan mechanisms.

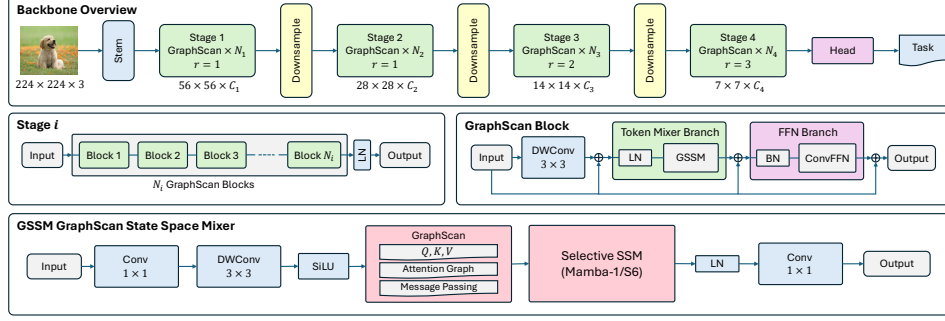


Figure 3: **GraphScan-Mamba architecture.** *Top*: four-stage hierarchical backbone with an overlapping convolutional stem to $1/4$ resolution and stage-wise downsampling, with stage-wise graph radii $r = (1, 1, 2, 3)$ yielding 3×3 , 3×3 , 5×5 , and 7×7 local neighborhoods. *Middle*: each of N_i blocks combines a depthwise positional branch, a GraphScan-SSM token-mixing branch, and a ConvFFN branch. *Bottom*: inside GSSM, features pass through 1×1 projection, 3×3 depthwise conv, SiLU, GraphScan, selective SSM, normalization, and output projection—a fixed-local $\rightarrow$ learned-local $\rightarrow$ global progression.

3.5 GraphScan-Mamba Architecture

We integrate GraphScan into a hierarchical Vision Mamba backbone, **GraphScan-Mamba**, following the standard four-stage design used by modern vision backbones [35, 22, 45, 68, 30]: an overlapping convolutional stem produces a $1/4$ -resolution feature map, and subsequent stages progressively downsample spatial resolution while increasing channel dimension. This architectural template is deliberately conventional; our goal is to isolate the effect of replacing geometric pre-SSM routing with graph-induced semantic routing. Fig. 3 summarizes the full backbone, the repeated GraphScan block, and the GSSM token mixer where GraphScan is inserted before the selective SSM.

Block structure. Each block has three residual components applied in sequence: $X \leftarrow X + \text{DWConv}_{3 \times 3}(X)$, then $X \leftarrow X + \text{GSSM}(\text{LN}(X))$, then $X \leftarrow X + \text{ConvFFN}(\text{BN}(X))$ (omitting drop path and layer-scale factors). The first is a residual depthwise convolution that injects local positional bias [4, 30], fused at implementation time into a single convolution with an identity-offset kernel. ConvFFN stacks a 1×1 pointwise convolution, a 3×3 depthwise convolution, and a final 1×1 pointwise convolution [62, 30].

GraphScan-SSM token mixer. GraphScan is inserted between the input stage and the selective SSM: $\tilde{X} = \text{SiLU}(\text{DWConv}_{3 \times 3}(\text{Proj}_{\text{in}}(X)))$, $X^{\text{gs}} = \text{GraphScan}(\tilde{X})$, and $\text{GSSM}(X) = \text{Proj}_{\text{out}}(\text{LN}(\text{SSM}(X^{\text{gs}})))$. Each token mixer therefore follows a *fixed-local* (depthwise convolution) $\rightarrow$ *learned-local* (GraphScan) $\rightarrow$ *global* (selective SSM) progression.

Stage-wise graph radius. The graph radius r grows with stage depth: smaller windows at high resolution, where local mixing suffices, and larger windows in low-resolution late stages, where the receptive field needs to become effectively global. At the final stage, the GraphScan neighborhood spans the entire feature map while remaining linear in the spatial budget. Exact radii, channel widths, depths, and training configurations are reported in Sec. 4.

4 Experiments

We evaluate GraphScan-Mamba on three standard recognition benchmarks: ImageNet-1K classification [7], COCO object detection and instance segmentation [31] with Mask R-CNN [20], and ADE20K semantic segmentation [76] with UperNet [69]. Across all three tasks, GraphScan-Mamba achieves state-of-the-art accuracy among Vision SSM backbones at comparable parameter and FLOP budgets. We also ablate pre-SSM placement, graph radius, relative positional bias, and head count; per-block attention and effective-routing visualizations of the trained backbone are reported in App. D.

Architecture variants. We instantiate three model sizes (Tiny, Small, Base), following the conventional channel-and-depth scaling axes used by hierarchical Vision Mamba backbones [35]. The macro topology, MLP ratios, and graph radii are shared across variants; only channel widths and stage depths vary. Radii follow $r = (1, 1, 2, 3)$, yielding 3×3 , 3×3 , 5×5 , and 7×7 neighborhoods at stages 1–4; the stage-4 neighborhood spans the full 7×7 feature map. Per-variant channel widths, depths, MLP ratios, parameters, and FLOPs are reported in App. C.

4.1 Image Classification on ImageNet-1K

Setup. We follow the standard DeiT-style training recipe [54] adopted across recent Vision Mamba works [35]. All variants are trained for 300 epochs at 224^2 resolution using AdamW with a cosine learning-rate schedule, 20-epoch linear warmup, base learning rate 1×10^{-3} , weight decay 5×10^{-2} , and batch size 1024. Augmentation and regularization include RandAugment, Repeated Augmentation, Mixup, CutMix, Random Erasing, label smoothing, and stochastic depth, with the maximum stochastic-depth rate linearly interpolated to $\{0.2, 0.3, 0.5\}$ for T/S/B. Center cropping is applied at evaluation. FLOPs and parameters are reported at 224^2 .

Results. As shown in Table 2, GraphScan-Mamba consistently outperforms ConvNet, Transformer, and Vision SSM backbones at every parameter scale. GraphScan-Mamba-B reaches 86.5% top-1 accuracy, +1.2 over the strongest Vision SSM at this scale (Spatial-Mamba-B at 85.3%) and +2.6 over VMamba-B at 83.9%. At the Small scale, GraphScan-Mamba-S obtains 85.7%, exceeding DAMamba-S (84.8%) by +0.9 and VMamba-S (83.6%) by +2.1. At Tiny, GraphScan-Mamba-T attains 84.4%, +0.6 above the strongest Vision SSM at this scale (DAMamba-T at 83.8%) and +1.9 above VMamba-T at 82.5%. The lead is preserved against the strongest ConvNets and Transformers at matched parameter budgets, including ConvNeXt-B (83.8%), Swin-B (83.5%), and NAT-B (84.3%). These results demonstrate that injecting a local semantic routing layer before the selective scan is an effective lever for surpassing strong ConvNet and Transformer backbones while preserving the linear scaling of state-space models.

Table 2: **Image classification on ImageNet-1K.** All entries use 224^2 input. “Type”: ConvNet (C), Transformer (T), State-Space Model (S), or Graph Neural Network (G). Best result per parameter regime is in **bold**.

Method	Type	Params	FLOPs	Top-1 (%)
ConvNeXt-T [37]	C	29M	4.5G	82.1
InceptionNeXt-T [74]	C	28M	4.2G	82.3
Swin-T [36]	T	29M	4.5G	81.3
NAT-T [17]	T	28M	4.3G	83.2
BiFormer-S [77]	T	26M	4.5G	83.8
PyramidViG-S [16]	G	27M	4.6G	82.1
VCMamba-M [40]	S	21M	2.3G	81.5
VCMamba-B [40]	S	32M	4.0G	82.6
Vim-S [78]	S	26M	5.1G	80.5
PlainMamba-L2 [72]	S	25M	8.1G	81.6
VMamba-T [35]	S	31M	4.9G	82.5
LocalVMamba-T [22]	S	26M	5.7G	82.7
MSVMamba-T [45]	S	33M	4.6G	82.8
MambaVision-T [18]	S	32M	4.4G	82.3
GroupMamba-T [44]	S	23M	4.5G	83.3
GroupMamba-S [44]	S	34M	7.0G	83.9
Spatial-Mamba-T [68]	S	27M	4.5G	83.5
VSSD-T [46]	S	24M	4.5G	83.7
DAMamba-T [30]	S	26M	4.8G	83.8
GraphScan-Mamba-T (ours)	S	28M	5.2G	84.4
ConvNeXt-S [37]	C	50M	8.7G	83.1
InceptionNeXt-S [74]	C	49M	8.4G	83.5
Swin-S [36]	T	50M	8.7G	83.0
NAT-S [17]	T	51M	7.8G	83.7
BiFormer-B [77]	T	57M	9.8G	84.3
PyramidViG-M [16]	G	52M	8.9G	83.1
PlainMamba-L3 [72]	S	50M	14.4G	82.3
VMamba-S [35]	S	50M	8.7G	83.6
LocalVMamba-S [22]	S	50M	11.4G	83.7
MambaVision-S [18]	S	50M	7.5G	83.3
VSSD-S [46]	S	40M	7.4G	84.1
Spatial-Mamba-S [68]	S	43M	7.1G	84.6
DAMamba-S [30]	S	45M	10.3G	84.8
GraphScan-Mamba-S (ours)	S	49M	11.1G	85.7
ConvNeXt-B [37]	C	89M	15.4G	83.8
InceptionNeXt-B [74]	C	87M	14.9G	84.0
Swin-B [36]	T	88M	15.4G	83.5
NAT-B [17]	T	90M	13.7G	84.3
PyramidViG-B [16]	G	93M	16.8G	83.7
GroupMamba-B [44]	S	57M	14.0G	84.5
VMamba-B [35]	S	89M	15.4G	83.9
MambaVision-B [18]	S	98M	15.0G	84.2
VSSD-B [46]	S	89M	16.1G	84.7
DAMamba-B [30]	S	86M	16.3G	85.2
Spatial-Mamba-B [68]	S	96M	15.8G	85.3
GraphScan-Mamba-B (ours)	S	93M	17.8G	86.5

4.2 Object Detection and Instance Segmentation on COCO

Setup. We adopt Mask R-CNN [20] with FPN, training on COCO train2017 and reporting on val2017 [31]. We use the standard $1 \times$ (12-epoch, single-scale) and $3 \times +MS$ (36-epoch, multi-scale) schedules with AdamW at initial learning rate 1×10^{-4} and batch size 16. Backbones are initialized from ImageNet-1K pretraining; the rest of the detector is randomly initialized. All experiments are implemented in MMDetection [38]. FLOPs are reported at 1280×800 input, and we report box AP (AP^b), AP_{50}^b , mask AP (AP^m), and AP_{50}^m for both schedules.

Results. Table 3 summarizes COCO performance. Under the $1 \times$ schedule, GraphScan-Mamba-T/S/B achieve box mAP of 49.1/50.7/51.9, outperforming VMamba-T/S/B by 1.8/2.0/2.7, Swin-T/S/B by 6.4/5.9/5.0, and ConvNeXt-T/S/B by 4.9/5.3/4.9. Mask mAP follows the same trend at 43.9/45.3/46.0, exceeding VMamba-T/S/B by 1.2/1.6/1.9. Under the longer $3 \times +MS$ schedule, GraphScan-Mamba-T/S/B reach 51.0/52.0/52.5 box mAP and 45.3/45.9/46.2 mask mAP, again leading the strongest Vision SSM at each scale. The gains transfer to dense prediction without modifying the backbone, indicating that the local semantic routing learned during classification yields tokens that remain useful when the network is evaluated against detection-scale spatial structure.

4.3 Semantic Segmentation on ADE20K

Setup. We adopt UperNet [69] on ADE20K [76], fine-tuning ImageNet-pretrained backbones for 160K iterations at 512×512 crops with AdamW and batch size 16. All experiments are implemented in MMSegmentation [39]. We report single-scale (SS) and multi-scale (MS) mIoU; FLOPs are reported at 512×2048 input.

Table 3: **COCO object detection and instance segmentation** with Mask R-CNN, $1\times$ and $3\times$ +MS schedules. “#P” is the full-detector parameter count.

Backbone	#P	Mask R-CNN $1\times$				Mask R-CNN $3\times$ +MS			
		AP ^b	AP ^b ₅₀	AP ^m	AP ^m ₅₀	AP ^b	AP ^b ₅₀	AP ^m	AP ^m ₅₀
Swin-T [36]	48M	42.7	65.2	39.3	62.2	46.0	68.1	41.6	65.1
ConvNeXt-T [37]	48M	44.2	66.6	40.1	63.3	46.2	67.9	41.7	65.0
NAT-T [17]	48M	—	—	—	—	47.7	69.0	42.6	66.1
BiFormer-S [77]	—	47.8	69.8	43.2	66.8	—	—	—	—
VMamba-T [35]	50M	47.3	69.3	42.7	66.4	48.8	70.4	43.7	67.4
LocalVMamba-T [22]	45M	46.7	68.7	42.2	65.7	48.7	70.1	43.4	67.0
MSVMamba-T [45]	53M	46.9	68.8	42.2	65.6	48.3	69.5	43.2	66.8
GroupMamba-T [44]	40M	47.6	69.8	42.9	66.5	—	—	—	—
VSSD-T [46]	44M	46.9	69.4	42.6	66.4	48.8	70.4	43.6	67.6
Spatial-Mamba-T [68]	46M	47.6	69.6	42.9	66.5	49.3	70.7	43.8	67.8
DAMamba-T [30]	45M	48.5	70.3	43.4	67.2	50.4	71.4	44.8	68.6
GraphScan-Mamba-T (ours)	47M	49.1	70.9	43.9	67.7	51.0	71.9	45.3	69.1
Swin-S [36]	69M	44.8	68.6	40.9	65.3	48.2	69.8	43.2	67.0
ConvNeXt-S [37]	70M	45.4	67.9	41.8	65.2	47.9	70.0	42.9	66.9
NAT-S [17]	70M	—	—	—	—	48.4	69.8	43.2	66.9
BiFormer-B [77]	—	48.6	70.5	43.7	67.6	—	—	—	—
VMamba-S [35]	70M	48.7	70.0	43.7	67.3	49.9	70.9	44.2	68.2
LocalVMamba-S [22]	69M	48.4	69.9	43.2	66.7	49.9	70.5	44.1	67.8
VSSD-S [46]	59M	48.4	70.1	43.5	67.2	—	—	—	—
Spatial-Mamba-S [68]	63M	49.2	70.8	44.0	67.9	50.5	71.5	44.6	68.7
DAMamba-S [30]	65M	49.8	71.2	44.5	68.4	51.2	72.1	45.1	69.2
GraphScan-Mamba-S (ours)	69M	50.7	72.1	45.3	69.1	52.0	72.8	45.9	69.9
Swin-B [36]	107M	46.9	69.2	42.3	66.0	48.6	70.0	43.3	67.1
ConvNeXt-B [37]	107M	47.0	69.4	42.7	66.3	48.5	70.1	43.5	67.1
VMamba-B [35]	108M	49.2	71.4	44.1	68.3	—	—	—	—
Spatial-Mamba-B [68]	115M	50.4	71.8	45.1	69.1	—	—	—	—
DAMamba-B [30]	105M	50.6	71.9	44.9	68.9	51.4	72.3	45.3	69.5
GraphScan-Mamba-B (ours)	112M	51.9	73.0	46.0	69.9	52.5	73.2	46.2	70.3

Table 4: **Semantic segmentation on ADE20K** with UperNet, 160K iters. “#P” and FLOPs at 512×2048 .

Backbone	#P	FLOPs	SS	MS
Swin-T [36]	60M	945G	44.4	45.8
ConvNeXt-T [37]	60M	939G	46.0	46.7
NAT-T [17]	58M	934G	47.1	48.4
BiFormer-S [77]	—	—	49.8	50.8
VMamba-T [35]	62M	949G	48.0	48.8
LocalVMamba-T [22]	57M	970G	47.9	49.1
MSVMamba-T [45]	65M	942G	47.6	48.5
GroupMamba-T [44]	—	—	48.6	49.2
VSSD-T [46]	53M	941G	47.9	48.7
Spatial-Mamba-T [68]	57M	936G	48.6	49.4
DAMamba-T [30]	55M	937G	50.3	51.2
GraphScan-Mamba-T (ours)	57M	942G	50.9	51.8
Swin-S [36]	81M	1039G	47.6	49.5
ConvNeXt-S [37]	82M	1027G	48.7	49.6
NAT-S [17]	82M	1010G	48.0	49.5
BiFormer-B [77]	—	—	51.0	51.7
VMamba-S [35]	82M	1028G	50.6	51.2
LocalVMamba-S [22]	81M	1095G	50.0	51.0
Spatial-Mamba-S [68]	73M	992G	50.6	51.4
DAMamba-S [30]	75M	1050G	51.2	52.0
GraphScan-Mamba-S (ours)	79M	1065G	52.1	52.9
Swin-B [36]	121M	1188G	48.1	49.7
ConvNeXt-B [37]	122M	1170G	49.1	49.9
NAT-B [17]	123M	1137G	48.5	49.7
VMamba-B [35]	122M	1170G	51.0	51.6
Spatial-Mamba-B [68]	127M	1176G	51.8	52.6
DAMamba-B [30]	117M	1178G	51.9	52.3
GraphScan-Mamba-B (ours)	124M	1210G	53.2	53.6

Results. Table 4 summarizes ADE20K accuracy. GraphScan-Mamba-T/S/B reach mIoU (SS) of 50.9/52.1/53.2, surpassing VMamba-T/S/B by 2.9/1.5/2.2 and the strongest Vision SSM baseline at each scale by $+0.6/+0.9/+1.3$. Multi-scale tests show analogous improvements (51.8/52.9/53.6). As discussed in Sec. 3.4, the local-global structure induced by GraphScan supplies semantically nearby content into the selective scan, which is especially valuable for scene-level segmentation, where two-dimensional adjacency frequently disagrees with raster order.

4.4 Ablation Studies

We ablate four design choices on GraphScan-Mamba-T with all training settings inherited from Sec. 4.1: pre- vs. post-SSM placement of the routing operator, graph radius schedule, the relative-position bias b_{rel} , and the number of affinity heads. Each ablation modifies a single factor while holding the rest at their default; the default configuration is marked with $\star$.

Table 5: **Ablation studies** on GraphScan-Mamba-T (training settings identical to Sec. 4.1). Default configuration marked with $\star$. Top-1 (%) reported.

(a) Routing placement		(b) Radius schedule		(c) Rel. pos. bias		(d) Affinity heads	
Placement	Top-1	(r_1, r_2, r_3, r_4)	Top-1	b_{rel}	Top-1	Heads	Top-1
None (base SSM)	82.4	(1, 1, 1, 1)	83.5	without	84.3	1 $\star$	84.4
Post-SSM	83.7	(1, 1, 2, 3) $\star$	84.4	with $\star$	84.4	2	84.1
Pre-SSM $\star$	84.4	(2, 2, 3, 3)	84.7			4	84.7

Discussion. (a) Pre-SSM placement is the strongest configuration: moving the operator post-SSM costs 0.7 pt and removing it entirely a further 1.3 pt, consistent with the three preconditioning pathways identified in Sec. 3.4. (b) The default growing-radius (1, 1, 2, 3) occupies the compute-accuracy Pareto front: a globally smaller schedule loses 0.9 pt and a globally larger one offers only $+0.3$ pt at noticeably higher FLOPs. (c) The relative-position bias gives a small but consistent gain, indicating feature-conditioned affinities benefit from a light geometric prior. (d) Multi-head GraphScan is non-monotonic ($H=2$ dips below $H=1$, $H=4$ surpasses both at no added parameter cost); we retain $H=1$ for parity with prior work and treat multi-head as a lightweight extension.

5 Conclusion

We introduced **GraphScan**, a local windowed graph attention operator that preconditions tokens before the selective state-space scan; in a hierarchical Vision Mamba backbone, this single change yields state-of-the-art accuracy among Vision SSMs on ImageNet-1K, COCO, and ADE20K at comparable parameter and FLOP budgets. Because GraphScan sits on the input side, it is orthogonal to recurrence-side advances [46, 68, 6, 27]; composing both directions is a natural next step. To the question posed in the title: yes—graphs do help Vision SSMs see better.

References

- [1] Peter W Battaglia, Jessica B Hamrick, Victor Bapst, Alvaro Sanchez-Gonzalez, Vinicius Zambaldi, Mateusz Malinowski, Andrea Tacchetti, David Raposo, Adam Santoro, Ryan Faulkner, et al. Relational inductive biases, deep learning, and graph networks. *arXiv preprint arXiv:1806.01261*, 2018.
- [2] Zhe Chen, Yuchen Duan, Wenhai Wang, Junjun He, Tong Lu, Jifeng Dai, and Yu Qiao. Vision transformer adapter for dense predictions. In *The Eleventh International Conference on Learning Representations*, 2023.
- [3] Xiangxiang Chu, Zhi Tian, Yuqing Wang, Bo Zhang, Haibing Ren, Xiaolin Wei, Huaxia Xia, and Chunhua Shen. Twins: Revisiting the design of spatial attention in vision transformers. *Advances in neural information processing systems*, 34:9355–9366, 2021.
- [4] Xiangxiang Chu, Zhi Tian, Bo Zhang, Xinlong Wang, and Chunhua Shen. Conditional positional encodings for vision transformers. In *The Eleventh International Conference on Learning Representations*, 2023.
- [5] Zihang Dai, Hanxiao Liu, Quoc V Le, and Mingxing Tan. Coatnet: Marrying convolution and attention for all data sizes. *Advances in neural information processing systems*, 34:3965–3977, 2021.
- [6] Tri Dao and Albert Gu. Transformers are ssms: generalized models and efficient algorithms through structured state space duality. In *Proceedings of the 41st International Conference on Machine Learning*. JMLR.org, 2024.
- [7] Jia Deng, Wei Dong, Richard Socher, Li-Jia Li, Kai Li, and Li Fei-Fei. Imagenet: A large-scale hierarchical image database. In *2009 IEEE Conference on Computer Vision and Pattern Recognition*, pages 248–255, 2009.
- [8] Xiaohan Ding, Yiyuan Zhang, Yixiao Ge, Sijie Zhao, Lin Song, Xiangyu Yue, and Ying Shan. Unireplknet: A universal perception large-kernel convnet for audio video point cloud time-series and image recognition. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 5513–5524, 2024.
- [9] Xiaoyi Dong, Jianmin Bao, Dongdong Chen, Weiming Zhang, Nenghai Yu, Lu Yuan, Dong Chen, and Baining Guo. Cswin transformer: A general vision transformer backbone with cross-shaped windows. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 12124–12134, 2022.
- [10] Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xiaohua Zhai, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer, Georg Heigold, Sylvain Gelly, et al. An image is worth 16x16 words: Transformers for image recognition at scale. *arXiv preprint arXiv:2010.11929*, 2020.
- [11] Daniel Y Fu, Tri Dao, Khaled Kamal Saab, Armin W Thomas, Atri Rudra, and Christopher Re. Hungry hungry hippos: Towards language modeling with state space models. In *The Eleventh International Conference on Learning Representations*, 2023.
- [12] Justin Gilmer, Samuel S Schoenholz, Patrick F Riley, Oriol Vinyals, and George E Dahl. Neural message passing for quantum chemistry. In *International conference on machine learning*, pages 1263–1272. Pmlr, 2017.
- [13] Albert Gu and Tri Dao. Mamba: Linear-time sequence modeling with selective state spaces. In *First conference on language modeling*, 2024.
- [14] Albert Gu, Karan Goel, Ankit Gupta, and Christopher Ré. On the parameterization and initialization of diagonal state space models. *Advances in neural information processing systems*, 35:35971–35983, 2022.
- [15] Albert Gu, Karan Goel, and Christopher Re. Efficiently modeling long sequences with structured state spaces. In *International Conference on Learning Representations*, 2022.
- [16] Kai Han, Yunhe Wang, Jianyuan Guo, Yehui Tang, and Enhua Wu. Vision gnn: An image is worth graph of nodes. *Advances in neural information processing systems*, 35:8291–8303, 2022.
- [17] Ali Hassani, Steven Walton, Jiachen Li, Shen Li, and Humphrey Shi. Neighborhood attention transformer. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 6185–6194, 2023.
- [18] Ali Hatamizadeh and Jan Kautz. Mambavision: A hybrid mamba-transformer vision backbone. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 25261–25270, 2025.
- [19] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 770–778, 2016.
- [20] Kaiming He, Georgia Gkioxari, Piotr Dollár, and Ross Girshick. Mask r-cnn. In *2017 IEEE International Conference on Computer Vision (ICCV)*, pages 2980–2988, 2017.
- [21] Gao Huang, Zhuang Liu, Laurens Van Der Maaten, and Kilian Q Weinberger. Densely connected convolutional networks. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 4700–4708, 2017.
- [22] Tao Huang, Xiaohuan Pei, Shan You, Fei Wang, Chen Qian, and Chang Xu. Localmamba: Visual state space model with windowed selective scan. In *European conference on computer vision*, pages 12–22. Springer, 2024.

- [23] R. E. Kalman. A new approach to linear filtering and prediction problems. *Journal of Basic Engineering*, 82(1):35–45, 1960.
- [24] Seungwook Kim and Minsu Cho. Similarity-aware selective state-space modeling for semantic correspondence. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV) Workshops*, pages 6147–6157, 2025.
- [25] Thomas N. Kipf and Max Welling. Semi-supervised classification with graph convolutional networks. In *International Conference on Learning Representations*, 2017.
- [26] Alex Krizhevsky, Ilya Sutskever, and Geoffrey E Hinton. Imagenet classification with deep convolutional neural networks. *Advances in neural information processing systems*, 25, 2012.
- [27] Aakash Lahoti, Kevin Li, Berlin Chen, Caitlin Wang, Aviv Bick, J Zico Kolter, Tri Dao, and Albert Gu. Mamba-3: Improved sequence modeling using state space principles. In *The Fourteenth International Conference on Learning Representations*, 2026.
- [28] Y. Lecun, L. Bottou, Y. Bengio, and P. Haffner. Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 86(11):2278–2324, 1998.
- [29] Shufan Li, Harkanwar Singh, and Aditya Grover. Mamba-nd: Selective state space modeling for multi-dimensional data. In *European Conference on Computer Vision*, pages 75–92. Springer, 2024.
- [30] Tanzhe Li, Caoshuo Li, Jiayi Lyu, Hongjuan Pei, Baochang Zhang, Taisong Jin, and Rongrong Ji. DAMamba: Vision state space model with dynamic adaptive scan. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems*, 2025.
- [31] Tsung-Yi Lin, Michael Maire, Serge Belongie, James Hays, Pietro Perona, Deva Ramanan, Piotr Dollár, and C. Lawrence Zitnick. Microsoft coco: Common objects in context. In *Computer Vision – ECCV 2014*, pages 740–755, Cham, 2014. Springer International Publishing.
- [32] Hanxiao Liu, Zihang Dai, David So, and Quoc V Le. Pay attention to mlps. *Advances in neural information processing systems*, 34:9204–9215, 2021.
- [33] Leiye Liu, Miao Zhang, Jihao Yin, Tingwei Liu, Wei Ji, Yongri Piao, and Huchuan Lu. Defmamba: Deformable visual state space model. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 8838–8847, 2025.
- [34] Shiwei Liu, Tianlong Chen, Xiaohan Chen, Xuxi Chen, Qiao Xiao, Boqian Wu, Tommi Kärkkäinen, Mykola Pechenizkiy, Decebal Constantin Mocanu, and Zhangyang Wang. More convnets in the 2020s: Scaling up kernels beyond 51x51 using sparsity. In *The Eleventh International Conference on Learning Representations*, 2023.
- [35] Yue Liu, Yunjie Tian, Yuzhong Zhao, Hongtian Yu, Lingxi Xie, Yaowei Wang, Qixiang Ye, Jianbin Jiao, and Yunfan Liu. Vmamba: Visual state space model. *Advances in neural information processing systems*, 37:103031–103063, 2024.
- [36] Ze Liu, Yutong Lin, Yue Cao, Han Hu, Yixuan Wei, Zheng Zhang, Stephen Lin, and Baining Guo. Swin transformer: Hierarchical vision transformer using shifted windows. In *Proceedings of the IEEE/CVF international conference on computer vision*, pages 10012–10022, 2021.
- [37] Zhuang Liu, Hanzi Mao, Chao-Yuan Wu, Christoph Feichtenhofer, Trevor Darrell, and Saining Xie. A convnet for the 2020s. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 11976–11986, 2022.
- [38] MMDetection Contributors. MMDetection. <https://github.com/open-mmlab/mmdetection>.
- [39] MMSegmentation Contributors. MMSegmentation. <https://github.com/open-mmlab/mmssegmentation>.
- [40] Mustafa Munir, Alex Zhang, and Radu Marculescu. Vcmamba: Bridging convolutions with multi-directional mamba for efficient visual representation. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV) Workshops*, pages 3037–3046, 2025.
- [41] Xiaohuan Pei, Tao Huang, and Chang Xu. Efficientvmamba: Atrous selective scan for light weight visual mamba. In *Proceedings of the AAAI conference on artificial intelligence*, pages 6443–6451, 2025.
- [42] Michael Poli, Stefano Massaroli, Eric Nguyen, Daniel Y Fu, Tri Dao, Stephen Baccus, Yoshua Bengio, Stefano Ermon, and Christopher Ré. Hyena hierarchy: Towards larger convolutional language models. In *International Conference on Machine Learning*, pages 28043–28078. PMLR, 2023.
- [43] Ilija Radosavovic, Raj Prateek Kosaraju, Ross Girshick, Kaiming He, and Piotr Dollár. Designing network design spaces. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 10428–10436, 2020.
- [44] Abdelrahman Shaker, Syed Talal Wasim, Salman Khan, Juergen Gall, and Fahad Shahbaz Khan. Group-mamba: Efficient group-based visual state space model. In *Proceedings of the Computer Vision and Pattern Recognition Conference*, pages 14912–14922, 2025.
- [45] Yuheng Shi, Minjing Dong, and Chang Xu. Multi-scale vmamba: Hierarchy in hierarchy visual state space model. *Advances in Neural Information Processing Systems*, 37:25687–25708, 2024.
- [46] Yuheng Shi, Mingjia Li, Minjing Dong, and Chang Xu. Vssd: Vision mamba with non-causal state space duality. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, pages 10819–10829, 2025.

- [47] Karen Simonyan and Andrew Zisserman. Very deep convolutional networks for large-scale image recognition. In *3rd International Conference on Learning Representations, ICLR 2015, San Diego, CA, USA, May 7-9, 2015, Conference Track Proceedings*. Association for Computing Machinery, 2015.
- [48] Jimmy T.H. Smith, Andrew Warrington, and Scott Linderman. Simplified state space layers for sequence modeling. In *The Eleventh International Conference on Learning Representations*, 2023.
- [49] Eduardo D Sontag. *Mathematical control theory: deterministic finite dimensional systems*. Springer Science & Business Media, 2013.
- [50] Christian Szegedy, Wei Liu, Yangqing Jia, Pierre Sermanet, Scott Reed, Dragomir Anguelov, Dumitru Erhan, Vincent Vanhoucke, and Andrew Rabinovich. Going deeper with convolutions. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 1–9, 2015.
- [51] Mingxing Tan and Quoc Le. Efficientnet: Rethinking model scaling for convolutional neural networks. In *International conference on machine learning*, pages 6105–6114. PMLR, 2019.
- [52] Lv Tang, HaoKe Xiao, Peng-Tao Jiang, Hao Zhang, Jinwei Chen, and Bo Li. Scalable visual state space model with fractal scanning. *arXiv preprint arXiv:2405.14480*, 2024.
- [53] Ilya O Tolstikhin, Neil Houlsby, Alexander Kolesnikov, Lucas Beyer, Xiaohua Zhai, Thomas Unterthiner, Jessica Yung, Andreas Steiner, Daniel Keysers, Jakob Uszkoreit, et al. Mlp-mixer: An all-mlp architecture for vision. *Advances in neural information processing systems*, 34:24261–24272, 2021.
- [54] Hugo Touvron, Matthieu Cord, Matthijs Douze, Francisco Massa, Alexandre Sablayrolles, and Hervé Jégou. Training data-efficient image transformers & distillation through attention. In *International conference on machine learning*, pages 10347–10357. PMLR, 2021.
- [55] Hugo Touvron, Piotr Bojanowski, Mathilde Caron, Matthieu Cord, Alaaeldin El-Nouby, Edouard Grave, Gautier Izacard, Armand Joulin, Gabriel Synnaeve, Jakob Verbeek, et al. Resmlp: Feedforward networks for image classification with data-efficient training. *IEEE transactions on pattern analysis and machine intelligence*, 45(4):5314–5321, 2022.
- [56] Zhengzhong Tu, Hossein Talebi, Han Zhang, Feng Yang, Peyman Milanfar, Alan Bovik, and Yinxiao Li. Maxvit: Multi-axis vision transformer. In *European conference on computer vision*, pages 459–479. Springer, 2022.
- [57] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. *Advances in neural information processing systems*, 30, 2017.
- [58] Petar Veličković, Guillem Cucurull, Arantxa Casanova, Adriana Romero, Pietro Liò, and Yoshua Bengio. Graph attention networks. In *International Conference on Learning Representations*, 2018.
- [59] Ao Wang, Hui Chen, Zijia Lin, Jungong Han, and Guiguang Ding. Repvit: Revisiting mobile cnn from vit perspective. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 15909–15920, 2024.
- [60] Bo Wang, Mengyuan Xu, Yue Yan, Yuqun Yang, Kechen Shu, Wei Ping, Xu Tang, Wei Jiang, and Zheng You. Asm-unet: Adaptive scan mamba integrating group commonalities and individual variations for fine-grained segmentation, 2025.
- [61] Wenhai Wang, Enze Xie, Xiang Li, Deng-Ping Fan, Kaitao Song, Ding Liang, Tong Lu, Ping Luo, and Ling Shao. Pyramid vision transformer: A versatile backbone for dense prediction without convolutions. In *Proceedings of the IEEE/CVF international conference on computer vision*, pages 568–578, 2021.
- [62] Wenhai Wang, Enze Xie, Xiang Li, Deng-Ping Fan, Kaitao Song, Ding Liang, Tong Lu, Ping Luo, and Ling Shao. Pvt v2: Improved baselines with pyramid vision transformer. *Computational visual media*, 8(3):415–424, 2022.
- [63] Wenhai Wang, Jifeng Dai, Zhe Chen, Zhenhang Huang, Zhiqi Li, Xizhou Zhu, Xiaowei Hu, Tong Lu, Lewei Lu, Hongsheng Li, et al. Internimage: Exploring large-scale vision foundation models with deformable convolutions. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 14408–14419, 2023.
- [64] Xiaolong Wang, Ross Girshick, Abhinav Gupta, and Kaiming He. Non-local neural networks. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 7794–7803, 2018.
- [65] Sanghyun Woo, Shoubhik Debnath, Ronghang Hu, Xinlei Chen, Zhuang Liu, In So Kweon, and Saining Xie. Convnext v2: Co-designing and scaling convnets with masked autoencoders. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 16133–16142, 2023.
- [66] JiaFu Wu, Jian Li, Jiangning Zhang, Boshen Zhang, Mingmin Chi, Yabiao Wang, and Chengjie Wang. Pvg: Progressive vision graph for vision recognition. In *Proceedings of the 31st ACM international conference on multimedia*, pages 2477–2486, 2023.
- [67] Zhuofan Xia, Xuran Pan, Shiji Song, Li Erran Li, and Gao Huang. Vision transformer with deformable attention. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 4794–4803, 2022.
- [68] Chaodong Xiao, Minghan Li, Zhengqiang ZHANG, Deyu Meng, and Lei Zhang. Spatial-mamba: Effective visual state space models via structure-aware state fusion. In *The Thirteenth International Conference on Learning Representations*, 2025.

- [69] Tete Xiao, Yingcheng Liu, Bolei Zhou, Yuning Jiang, and Jian Sun. Unified perceptual parsing for scene understanding. In *Proceedings of the European Conference on Computer Vision (ECCV)*, 2018.
- [70] Fei Xie, Weijia Zhang, Zhongdao Wang, and Chao Ma. Quadmamba: Learning quadtree-based selective scan for visual state space model. In *The Thirty-eighth Annual Conference on Neural Information Processing Systems*, 2024.
- [71] Fei Xie, Zhongdao Wang, Weijia Zhang, and Chao Ma. Pvmamba: Parallelizing vision mamba via dynamic state aggregation. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, pages 10218–10228, 2025.
- [72] Chenhongyi Yang, Zehui Chen, Miguel Espinosa, Linus Ericsson, Zhenyu Wang, Jiaming Liu, and Elliot J. Crowley. Plainmamba: Improving non-hierarchical mamba in visual recognition. In *35th British Machine Vision Conference 2024, BMVC 2024, Glasgow, UK, November 25-28, 2024*. BMVA, 2024.
- [73] Jianwei Yang, Chunyuan Li, Pengchuan Zhang, Xiyang Dai, Bin Xiao, Lu Yuan, and Jianfeng Gao. Focal self-attention for local-global interactions in vision transformers. *arXiv preprint arXiv:2107.00641*, 2021.
- [74] Weihao Yu, Pan Zhou, Shuicheng Yan, and Xinchao Wang. Inceptionnext: When inception meets convnext. In *Proceedings of the IEEE/cvf conference on computer vision and pattern recognition*, pages 5672–5683, 2024.
- [75] Kai Zhang, Xia Yuan, and Chunxia Zhao. Objectness scan for efficient vision mamba. *Knowledge-Based Systems*, 330:114573, 2025.
- [76] Bolei Zhou, Hang Zhao, Xavier Puig, Sanja Fidler, Adela Barriuso, and Antonio Torralba. Scene parsing through ade20k dataset. In *2017 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 5122–5130, 2017.
- [77] Lei Zhu, Xinjiang Wang, Zhanghan Ke, Wayne Zhang, and Rynson WH Lau. Biformer: Vision transformer with bi-level routing attention. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 10323–10333, 2023.
- [78] Lianghui Zhu, Bencheng Liao, Qian Zhang, Xinlong Wang, Wenyu Liu, and Xinggang Wang. Vision mamba: Efficient visual representation learning with bidirectional state space model. In *Forty-first International Conference on Machine Learning*, 2024.

A Analysis of GraphScan-Induced Selective Scan

This appendix provides a more detailed analysis of how GraphScan modifies the selective scan computation. The goal is not to prove a task-level generalization guarantee, but to make precise the architectural effect of routing tokens before the SSM. GraphScan changes both the values entering the recurrent state and the token-dependent selective parameters that control propagation and readout.

Throughout this appendix, we omit batch dimensions and use the notation from Sec. 3.1. All decompositions below are exact unless explicitly stated otherwise.

A.1 Selective Scan as a Token-Conditioned Dynamical System

For a visual token sequence $\mathbf{X} = [\mathbf{x}_1, \dots, \mathbf{x}_L]$, the Mamba-style selective scan can be written as a token-conditioned dynamical system:

$$\mathbf{H}_t = \bar{\mathbf{A}}_t(\mathbf{x}_t) \odot \mathbf{H}_{t-1} + \bar{\mathbf{B}}_t(\mathbf{x}_t) \odot \mathbf{x}_t^{\uparrow s}, \quad \mathbf{y}_t = \langle \mathbf{H}_t, \mathbf{c}_t(\mathbf{x}_t)^{\uparrow D} \rangle_{N_s}. \quad (10)$$

Here $\bar{\mathbf{A}}_t$ and $\bar{\mathbf{B}}_t$ are post-discretization tensors, while $\mathbf{c}_t$ is the input-dependent readout. The dependence on $\mathbf{x}_t$ includes the generation of the final positive step size Δ_t , the selective input vector, and the readout vector. Thus, unlike a linear time-invariant SSM, the recurrence coefficients are functions of the token sequence [13].

For a zero initial state, the hidden state unrolls as

$$\mathbf{H}_t = \sum_{i=1}^t \mathbf{G}_{t,i} \odot \bar{\mathbf{B}}_i(\mathbf{x}_i) \odot \mathbf{x}_i^{\uparrow s}, \quad \mathbf{G}_{t,i} = \bigodot_{j=i+1}^t \bar{\mathbf{A}}_j(\mathbf{x}_j). \quad (11)$$

This expression makes the sequence geometry explicit: information from token i reaches token t through the product of the intermediate state transitions along the scan path.

A.2 Exact Routed-vs-Base Decomposition

GraphScan replaces each token $\mathbf{x}_t$ with a routed token $\mathbf{x}'_t = \mathbf{x}_t + \mathbf{m}_t$, where, with $M = W_v W_o$ as in Eq. (4),

$$\mathbf{m}_t = \sum_{j \in \mathcal{S}_r(t)} \alpha_{tj} \mathbf{x}_j M. \quad (12)$$

The routed selective parameters are

$$\bar{\mathbf{A}}'_t = \bar{\mathbf{A}}_t(\mathbf{x}'_t), \quad \bar{\mathbf{B}}'_t = \bar{\mathbf{B}}_t(\mathbf{x}'_t), \quad \mathbf{c}'_t = \mathbf{c}_t(\mathbf{x}'_t),$$

and the routed recurrence is

$$\mathbf{H}'_t = \bar{\mathbf{A}}'_t \odot \mathbf{H}'_{t-1} + \bar{\mathbf{B}}'_t \odot (\mathbf{x}'_t)^{\uparrow s}. \quad (13)$$

Proposition A.1 (Exact routed-vs-base decomposition). Let $\delta \mathbf{H}_t = \mathbf{H}'_t - \mathbf{H}_t$ and define

$$\delta \bar{\mathbf{A}}_t = \bar{\mathbf{A}}'_t - \bar{\mathbf{A}}_t, \quad \delta \bar{\mathbf{B}}_t = \bar{\mathbf{B}}'_t - \bar{\mathbf{B}}_t, \quad \delta \mathbf{c}_t = \mathbf{c}'_t - \mathbf{c}_t.$$

Assuming the base and routed recurrences share the same initial state, $\delta \mathbf{H}_0 = 0$, we have

$$\delta \mathbf{H}_t = \sum_{i=1}^t \mathbf{G}'_{t,i} \odot \left[\bar{\mathbf{B}}'_i \odot \mathbf{m}_i^{\uparrow s} + \delta \bar{\mathbf{B}}_i \odot \mathbf{x}_i^{\uparrow s} + \delta \bar{\mathbf{A}}_i \odot \mathbf{H}_{i-1} \right], \quad (14)$$

where $\mathbf{G}'_{t,i} = \bigodot_{j=i+1}^t \bar{\mathbf{A}}'_j$ and $\mathbf{G}'_{t,t} = \mathbf{1}$. Moreover,

$$\delta \mathbf{y}_t = \langle \delta \mathbf{H}_t, (\mathbf{c}'_t)^{\uparrow D} \rangle_{N_s} + \langle \mathbf{H}_t, \delta \mathbf{c}_t^{\uparrow D} \rangle_{N_s}. \quad (15)$$

Proof. Subtract the base recurrence from the routed recurrence. Using $\mathbf{x}'_t = \mathbf{x}_t + \mathbf{m}_t$, $\mathbf{H}'_{t-1} = \mathbf{H}_{t-1} + \delta \mathbf{H}_{t-1}$, $\bar{\mathbf{A}}'_t = \bar{\mathbf{A}}_t + \delta \bar{\mathbf{A}}_t$, and $\bar{\mathbf{B}}'_t = \bar{\mathbf{B}}_t + \delta \bar{\mathbf{B}}_t$, we obtain

$$\begin{aligned} \delta \mathbf{H}_t &= \bar{\mathbf{A}}'_t \odot (\mathbf{H}_{t-1} + \delta \mathbf{H}_{t-1}) + \bar{\mathbf{B}}'_t \odot (\mathbf{x}_t + \mathbf{m}_t)^{\uparrow s} - \bar{\mathbf{A}}_t \odot \mathbf{H}_{t-1} - \bar{\mathbf{B}}_t \odot \mathbf{x}_t^{\uparrow s} \\ &= \bar{\mathbf{A}}'_t \odot \delta \mathbf{H}_{t-1} + (\bar{\mathbf{A}}'_t - \bar{\mathbf{A}}_t) \odot \mathbf{H}_{t-1} + \bar{\mathbf{B}}'_t \odot \mathbf{m}_t^{\uparrow s} + (\bar{\mathbf{B}}'_t - \bar{\mathbf{B}}_t) \odot \mathbf{x}_t^{\uparrow s}. \end{aligned}$$

Therefore,

$$\delta \mathbf{H}_t = \bar{\mathbf{A}}'_t \odot \delta \mathbf{H}_{t-1} + \delta \bar{\mathbf{A}}_t \odot \mathbf{H}_{t-1} + \bar{\mathbf{B}}'_t \odot \mathbf{m}_t^{\uparrow s} + \delta \bar{\mathbf{B}}_t \odot \mathbf{x}_t^{\uparrow s}.$$

Unrolling this recurrence from $\delta \mathbf{H}_0 = 0$ gives Eq. (14). For the output, write $\mathbf{H}'_t = \mathbf{H}_t + \delta \mathbf{H}_t$ and $\mathbf{c}'_t = \mathbf{c}_t + \delta \mathbf{c}_t$. Then

$$\begin{aligned} \delta \mathbf{y}_t &= \langle \mathbf{H}'_t, (\mathbf{c}'_t)^{\uparrow D} \rangle_{N_s} - \langle \mathbf{H}_t, \mathbf{c}_t^{\uparrow D} \rangle_{N_s} \\ &= \langle \mathbf{H}_t + \delta \mathbf{H}_t, (\mathbf{c}'_t)^{\uparrow D} \rangle_{N_s} - \langle \mathbf{H}_t, \mathbf{c}_t^{\uparrow D} \rangle_{N_s} \\ &= \langle \delta \mathbf{H}_t, (\mathbf{c}'_t)^{\uparrow D} \rangle_{N_s} + \langle \mathbf{H}_t, (\mathbf{c}'_t - \mathbf{c}_t)^{\uparrow D} \rangle_{N_s}. \end{aligned}$$

This proves Eq. (15). $\square$

Interpretation. Eq. (14) separates three hidden-state pathways: direct routed value injection, write modulation, and transition modulation. Eq. (15) adds a fourth pathway: readout modulation. The decomposition is exact and does not rely on treating the GraphScan message as small.

A.3 Local-Global Kernel Induced by GraphScan

We now isolate the direct routed value path in Eq. (14). Define the routed SSM input-to-output operator

$$\mathcal{K}'_{t,i}(\mathbf{z}) = \langle \mathbf{G}'_{t,i} \odot \bar{\mathbf{B}}'_i \odot \mathbf{z}^{\uparrow s}, (\mathbf{c}'_t)^{\uparrow D} \rangle_{N_s}, \quad i \leq t. \quad (16)$$

For $i > t$, set $\mathcal{K}'_{t,i}(\mathbf{z}) = \mathbf{0}$.

Corollary A.2 (Local-global kernel composition). The direct routed value contribution to $\delta \mathbf{y}_t$ is

$$\delta \mathbf{y}_t^{\text{route}} = \sum_{j=1}^L \sum_{\substack{i \leq t: \\ j \in \mathcal{S}_r(i)}} \alpha_{ij} \mathcal{K}'_{t,i}(\mathbf{x}_j M). \quad (17)$$

Proof. The direct routed value pathway in Eq. (14) contributes

$$\delta \mathbf{y}_t^{\text{route}} = \left\langle \sum_{i=1}^t \mathbf{G}'_{t,i} \odot \bar{\mathbf{B}}'_i \odot \mathbf{m}_i^{\uparrow s}, (\mathbf{c}'_t)^{\uparrow D} \right\rangle_{N_s}.$$

By the definition of $\mathcal{K}'_{t,i}$, this equals

$$\delta \mathbf{y}_t^{\text{route}} = \sum_{i=1}^t \mathcal{K}'_{t,i}(\mathbf{m}_i).$$

Substituting $\mathbf{m}_i = \sum_{j \in \mathcal{S}_r(i)} \alpha_{ij} \mathbf{x}_j M$ gives

$$\delta \mathbf{y}_t^{\text{route}} = \sum_{i=1}^t \mathcal{K}'_{t,i} \left(\sum_{j \in \mathcal{S}_r(i)} \alpha_{ij} \mathbf{x}_j M \right).$$

For fixed routed coefficients, $\mathcal{K}'_{t,i}$ is linear in its argument. Therefore,

$$\delta \mathbf{y}_t^{\text{route}} = \sum_{i=1}^t \sum_{j \in \mathcal{S}_r(i)} \alpha_{ij} \mathcal{K}'_{t,i}(\mathbf{x}_j M).$$

Exchanging the finite sums over i and j gives Eq. (17). $\square$

Interpretation. Eq. (17) shows that GraphScan composes two kernels. The first is a local semantic kernel α_{ij} defined over a bounded two-dimensional neighborhood. The second is a global selective SSM kernel $\mathcal{K}'_{t,i}$ defined along the one-dimensional scan path. Thus, GraphScan does not replace the SSM with local attention; it composes local semantic routing with global state-space propagation.

A.4 Scan-Distance Shortcuts under Bounded Survival

The local-global kernel clarifies why routing can help when two-dimensional adjacency is poorly aligned with raster order.

Proposition A.3 (Semantic shortcut under bounded routed survival). Assume the routed selective transitions satisfy $\|\bar{\mathbf{A}}'_j\|_\infty \leq \rho < 1$, and assume $\|\bar{\mathbf{B}}'_i\|_\infty \leq B_{\max}$ and $\|\mathbf{c}'_t\|_\infty \leq C_{\max}$. Then, for any $\mathbf{z} \in \mathbb{R}^D$ and any $i \leq t$,

$$\|\mathcal{K}'_{t,i}(\mathbf{z})\|_\infty \leq N_s B_{\max} C_{\max} \rho^{t-i} \|\mathbf{z}\|_\infty. \quad (18)$$

Consequently, a source token routed through slot i contributes to output t with attenuation governed by the scan distance $t - i$.

Proof. By definition,

$$\mathbf{G}'_{t,i} = \bigodot_{\ell=i+1}^t \bar{\mathbf{A}}'_\ell.$$

Using submultiplicativity of the elementwise ℓ_∞ norm under Hadamard products and the assumption $\|\bar{\mathbf{A}}'_\ell\|_\infty \leq \rho$, we obtain

$$\|\mathbf{G}'_{t,i}\|_\infty \leq \prod_{\ell=i+1}^t \|\bar{\mathbf{A}}'_\ell\|_\infty \leq \rho^{t-i}.$$

For each output channel d , the d -th component of $\mathcal{K}'_{t,i}(\mathbf{z})$ is

$$[\mathcal{K}'_{t,i}(\mathbf{z})]_d = \sum_{n=1}^{N_s} [\mathbf{G}'_{t,i}]_{d,n} [\bar{\mathbf{B}}'_i]_{d,n} z_d [\mathbf{c}'_t]_n.$$

Taking absolute values and applying the bounds on $\mathbf{G}'_{t,i}$, $\bar{\mathbf{B}}'_i$, $\mathbf{c}'_t$, and $\mathbf{z}$ gives

$$|[\mathcal{K}'_{t,i}(\mathbf{z})]_d| \leq \sum_{n=1}^{N_s} \rho^{t-i} B_{\max} C_{\max} \|\mathbf{z}\|_\infty = N_s B_{\max} C_{\max} \rho^{t-i} \|\mathbf{z}\|_\infty.$$

Taking the maximum over channels d proves Eq. (18). $\square$

Interpretation. This result does not claim GraphScan always improves accuracy. It shows a structural effect: a source patch can influence the SSM through a routed slot whose scan distance to the output may differ from the source patch's raster distance. When two-dimensional adjacency and one-dimensional raster order disagree, GraphScan supplies additional local semantic paths into the selective scan.

A.5 Reachability View from State-Space Systems

If the selective coefficients are frozen, Eq. (10) becomes a linear time-varying state-space system. Classical control theory studies how inputs enter such systems through transition operators and input matrices, often through reachability or controllability analyses [23, 49]. GraphScan modifies the input directions before they are injected into the recurrent state. In the frozen-coefficient view, the base system is driven by isolated raster tokens, while the routed system is driven by locally aggregated semantic neighborhoods.

More explicitly, with frozen coefficients and zero initial state, the base hidden state is

$$\mathbf{H}_t = \sum_{i=1}^t \mathbf{G}_{t,i} \odot \bar{\mathbf{B}}_i \odot \mathbf{x}_i^{\uparrow s}. \quad (19)$$

With GraphScan-routed inputs but the same frozen coefficients, the hidden state becomes

$$\mathbf{H}_t^{\text{gs}} = \sum_{i=1}^t \mathbf{G}_{t,i} \odot \bar{\mathbf{B}}_i \odot \left(\mathbf{x}_i + \sum_{j \in \mathcal{S}_r(i)} \alpha_{ij} \mathbf{x}_j M \right)^{\uparrow s}. \quad (20)$$

Eq. (20) shows that GraphScan changes the finite-horizon input-to-state map by introducing cross-token input directions induced by the sparse routing matrix $P(\mathbf{X})$. The recurrent state is no longer

driven only by isolated raster tokens; it is also driven by local semantic combinations of nearby tokens.

This reachability view should be interpreted carefully: GraphScan does not guarantee better controllability or generalization in the formal control-theoretic sense without additional assumptions. Instead, it provides a structural mechanism by which the finite recurrent state receives richer local semantic inputs before global propagation.

A.6 Baseline Containment

Proposition A.4 (Baseline containment). GraphScan-Mamba contains the corresponding base Vision Mamba model as a subnetwork.

Proof. From Eq. (3), setting $W_o = 0$ makes the routed message vanish for every token, so $\mathbf{x}'_i = \mathbf{x}_i$. If an implementation includes a bias in the output projection, setting that bias to zero as well gives the same identity map. Equivalently, Eq. (4) reduces to $\mathbf{X}' = \mathbf{X}$. The subsequent selective SSM, residual branches, feed-forward layers, and classifier are then identical to those of the corresponding base Vision Mamba model. Therefore, the base model is obtained as a parameter setting of GraphScan-Mamba. $\square$

Interpretation. This containment result does not imply optimization will always find a better solution, but it shows that GraphScan preserves the base model as a special case while adding trainable local routing paths before the SSM.

A.7 Relation to Local Attention and Vision GNNs

GraphScan uses a primitive related to local neighborhood attention and graph message passing. Neighborhood attention restricts $Q/K/V$ attention to a sliding neighborhood around each token, providing an efficient local attention mechanism for hierarchical Transformer backbones [17]; non-local networks generalize visual feature aggregation through pairwise interactions [64]. Vision GNNs instead view image patches as graph nodes and perform learned neighborhood aggregation over visual tokens [16, 66], building on the broader graph-network and message-passing primitives studied in deep learning [1, 12]. GraphScan borrows this local affinity aggregation primitive, but uses it in a different role: it is not the main token mixer of a Transformer or GNN backbone. It is a scan-time preconditioner placed immediately before the selective SSM. As shown above, this placement causes local semantic routing to influence both the values entering the recurrent state and the token-conditioned parameters of the selective scan.

B Extended Related Work

This appendix expands the condensed related work in Sec. 2 with the fuller treatment across all four prior-work paradigms.

B.1 Vision Backbones

Visual backbone design has evolved through several representation paradigms. Convolutional neural networks model images as regular grids and encode locality through sliding filters, hierarchical downsampling, residual connections, scaling rules, and large-kernel convolutional blocks [28, 26, 47, 50, 19, 21, 51, 43, 37, 65, 34, 8, 74]. Vision Transformers recast images as sequences of patch tokens and use self-attention for flexible long-range interaction [57, 10, 54]. To support high-resolution recognition and dense prediction, later Transformer backbones introduced hierarchical designs, shifted or windowed attention, pyramid structures, cross-shaped and focal attention, neighborhood attention, deformable attention, bi-level routing, non-local interactions, and convolution-attention hybrids [36, 61, 62, 3, 9, 73, 17, 67, 64, 77, 56, 5, 2, 63]. Beyond convolution and attention, all-MLP models explore token mixing with feed-forward layers [53, 55, 32], while Vision GNNs view image patches as graph nodes and propagate information through learned neighborhood aggregation [25, 58, 1, 16, 66].

GraphScan is related to this graph-based view, but it is not a standalone Vision GNN backbone. Instead, it uses lightweight local graph routing inside a Vision SSM, enriching visual tokens with semantic neighborhood context before global state-space mixing.

B.2 State Space Sequence Models

Structured state space models provide efficient sequence modeling with long-range interactions and linear or near-linear scaling in sequence length [15, 14, 48]. Early SSM-based architectures such as H3 and Hyena demonstrated the promise of long convolutional and state-space sequence mixers as alternatives to attention [11, 42]. Mamba introduced selective state spaces, making key SSM parameters input-dependent so that the model can selectively propagate, reset, or ignore information along the sequence while retaining efficient scan-based computation [13]. Mamba-2 further connected SSMs and attention through structured state space duality and improved the efficiency of the core sequence mixer [6]. Mamba-3 extends this line with recurrence-level improvements such as exponential-trapezoidal discretization, complex-valued state transitions, and MIMO updates [27].

These works improve the sequence model itself. Our work addresses a complementary question that arises when such models are used for vision: how should a two-dimensional visual feature map be routed before it is processed by a one-dimensional state-space operator?

B.3 Vision Mamba and Scan Design

Vision Mamba models adapt selective SSMs to images by converting feature maps into token sequences. Vim applies bidirectional Mamba blocks to visual sequences, while VMamba introduces a two-dimensional selective scan to process visual features along multiple spatial directions [78, 35]. Since scan order determines how two-dimensional structure is exposed to the one-dimensional recurrence, subsequent work has explored increasingly structured scan patterns. Continuous scans improve spatial continuity [72]; local-window scans preserve short-range dependencies and search layer-wise scan choices [22]; atrous and multi-scale scans improve efficiency and hierarchical coverage [41, 45]; grouped and multidimensional scans distribute scanning across channels or spatial axes [44, 29]; and fractal or space-filling scans aim to preserve locality under serialization [52].

These methods show that scan design is central to Vision SSM performance. However, most of them rely on predefined traversal patterns or layer-level choices, rather than input-dependent semantic routing.

B.4 Dynamic and Relational Scanning

Recent work investigates adaptive scanning for Vision SSMs. Quadtree-based scan methods learn spatial partitions with varying granularity [70]. Coordinate-offset methods predict deformable sampling locations and construct routed tokens through bilinear interpolation before feeding them to the SSM [30, 33]. Other approaches reorder or prioritize tokens using task- or content-dependent scores, or use similarity signals in specialized correspondence settings [75, 60, 24]. PVMamba modifies hidden-state generation through dynamic state aggregation and spatial sampling to reduce sequential constraints [71]. Spatial-Mamba introduces structure-aware fusion in the state space [68], VSSD reformulates SSD into a non-causal visual operator [46], and MambaVision hybridizes Mamba and Transformer blocks for high-level visual modeling [18].

GraphScan occupies a distinct design space. Rather than changing the SSM recurrence, collapsing scan order, or predicting continuous coordinates, GraphScan learns a local semantic graph over visual tokens and performs one-step message passing before the SSM. It is input-adaptive like dynamic scans, spatially bounded like local scans, and semantic like graph message passing. Consequently, the global selective SSM receives tokens that are already locally grounded, while preserving token count and linear scaling.

C Architecture Variants

Table 6 reports per-variant channel widths, stage depths, MLP ratios, graph radii, parameters, and FLOPs for the three GraphScan-Mamba scales used throughout the experiments.

Table 6: **GraphScan-Mamba variants.** Channels (C_1, C_2, C_3, C_4) , depths (N_1, N_2, N_3, N_4) , MLP ratios (m_1, m_2, m_3, m_4) , and graph radii (r_1, r_2, r_3, r_4) . Parameters and FLOPs reported at 224×224 input.

Variant	Channels	Depths	MLP ratios	Radii	Params	FLOPs
GraphScan-Mamba-T	80/160/320/512	3/4/12/5	4/4/3/3	1/1/2/3	28M	5.2G
GraphScan-Mamba-S	96/192/384/512	4/8/20/6	4/4/3/3	1/1/2/3	49M	11.1G
GraphScan-Mamba-B	112/224/448/640	4/8/25/8	4/4/3/4	1/1/2/3	93M	17.8G

D Visualizations

Capturing the per-block attention weights of the trained GraphScan-Mamba-S backbone surfaces two patterns. (i) *Stage-3 outer-ring specialization* (Fig. 4): the depthwise 3×3 convolution that runs before GraphScan already aggregates the inner 3×3 block into the query, so the operator suppresses the inner ring by roughly $40\times$ below uniform and attends almost exclusively to the $L_\infty=2$ outer ring—specializing in cells the preceding convolution cannot reach. (ii) *Stage-4 convergent global routing* (Fig. 5): with the local window covering the full 7×7 feature map, the displacement field $\Delta\hat{\mathbf{p}}_i = \sum_j \alpha_{ij}(\mathbf{p}_j - \mathbf{p}_i)$ converges onto salient object content and the effective scan path collapses there rather than tracing the spatial grid.

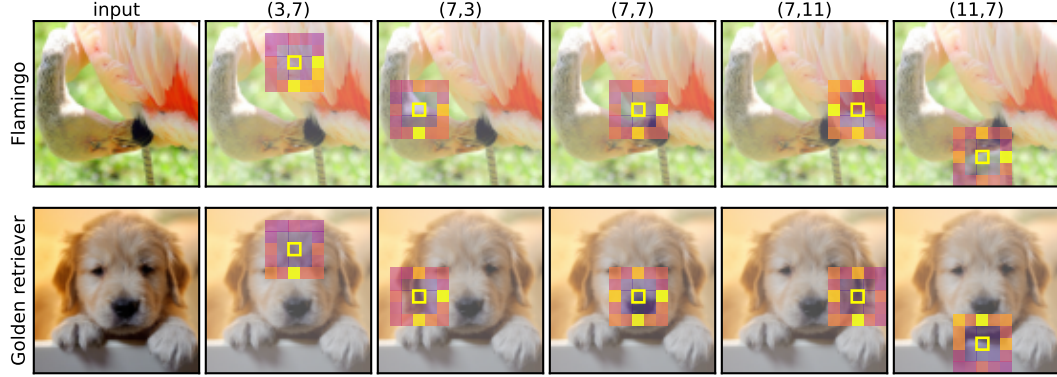


Figure 4: **Stage-3 GraphScan attention** on two ImageNet val images. For each query patch (yellow box), the 14×14 attention heatmap concentrates on the $L_\infty=2$ outer ring; the central 3×3 block is suppressed because the preceding depthwise convolution already covers it.

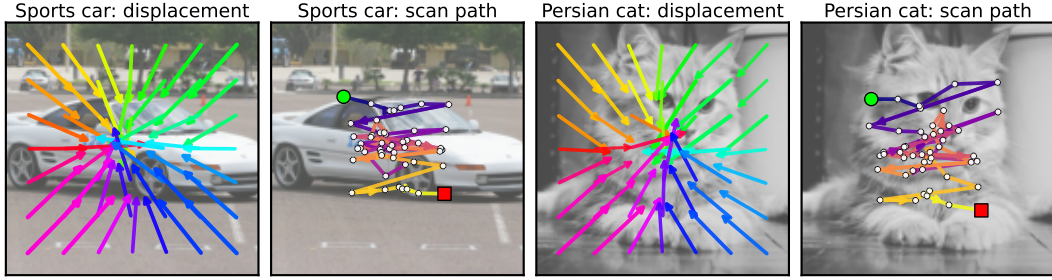


Figure 5: **Stage-4 effective routing**. Per-patch displacement field (HSV by direction) and the polyline visiting displaced positions in raster order (plasma, start $\bullet$, end $\blacksquare$). Both converge onto the discriminative object content.